Linear Regression

Linear model, gradient descent, feature expansion

Data Analytics and Mining, 2026

Fedor Shlyakhov
National Research University Higher School of Economics





November 6, 2026

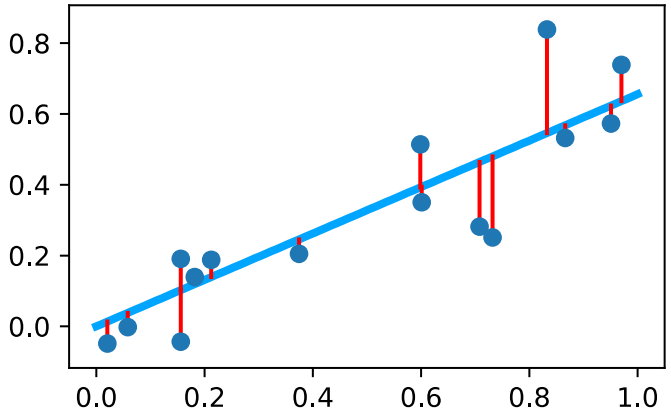
Why study linear models?



Linear models in a nutshell

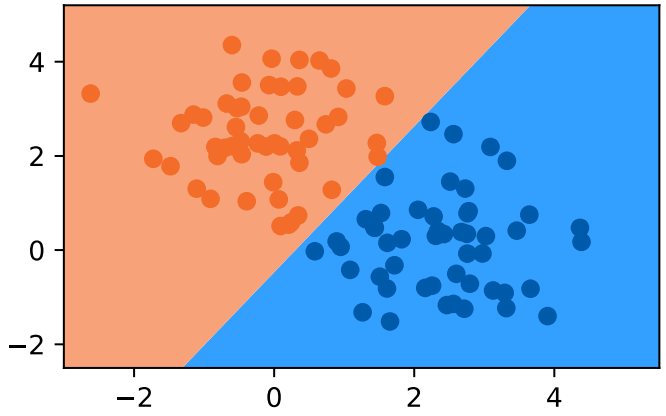
Regression:

$$\hat{f}(x) = \theta^{\mathrm{T}} x$$



Classification:

$$\hat{f}(x) = \mathbb{I}[\theta^{\mathrm{T}} x > 0]$$

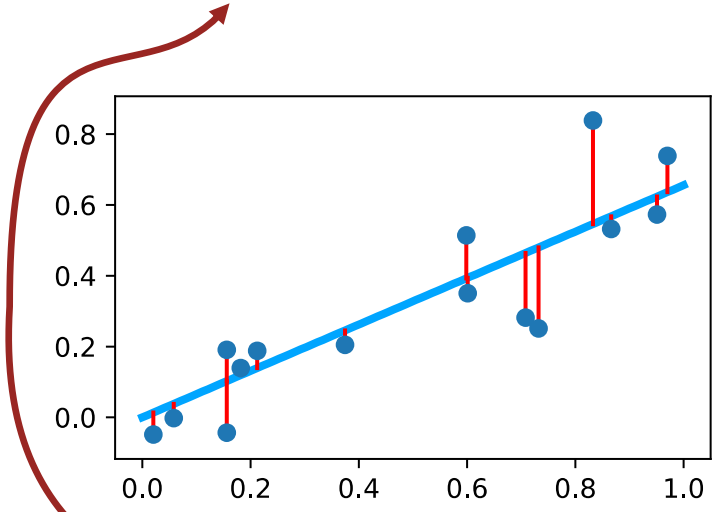


Outputs linear in inputs

Linear models in a nutshell

Regression:

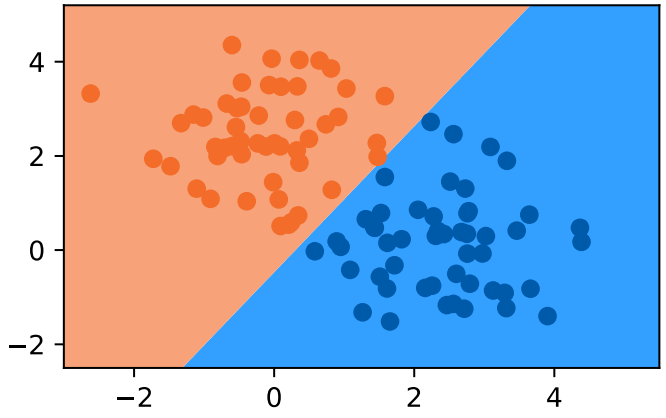
$$\hat{f}(x) = \theta^{\mathrm{T}} x$$



Targets $\in \mathbb{R}$ (or even $\mathbb{R}^m$ in the multidimensional case)

Classification:

$$\hat{f}(x) = \mathbb{I}[\theta^{\mathrm{T}} x > 0]$$

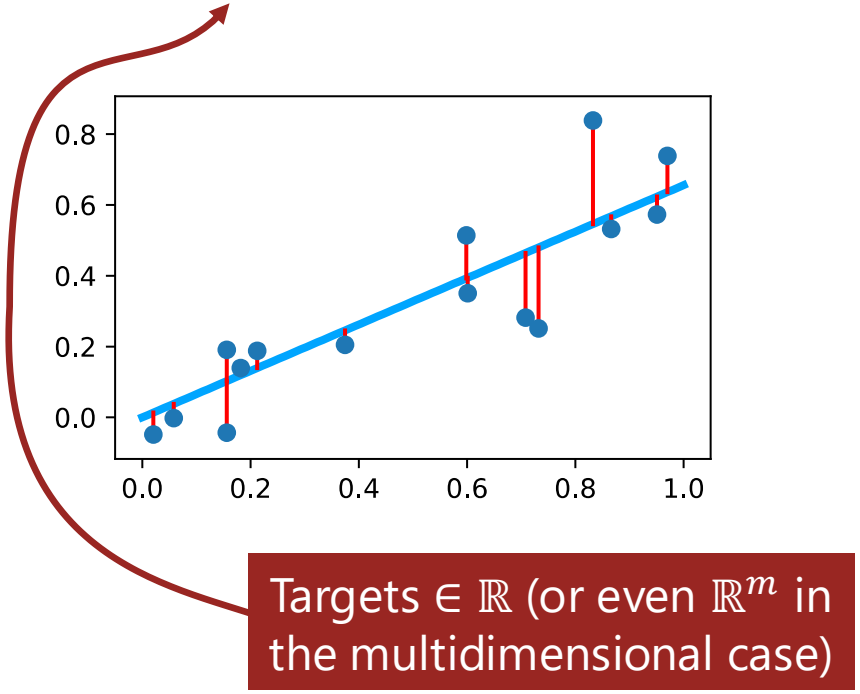


Outputs linear in inputs

Linear models in a nutshell

Regression:

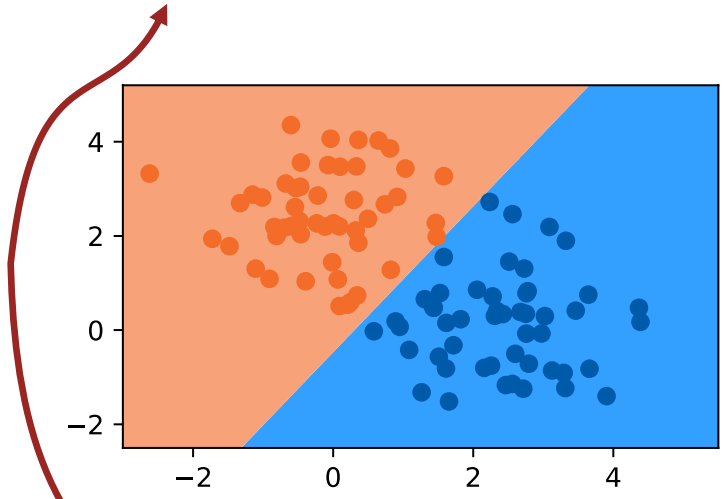
$$\hat{f}(x) = \theta^{\mathrm{T}} x$$



Targets $\in \mathbb{R}$ (or even $\mathbb{R}^m$ in the multidimensional case)

Classification:

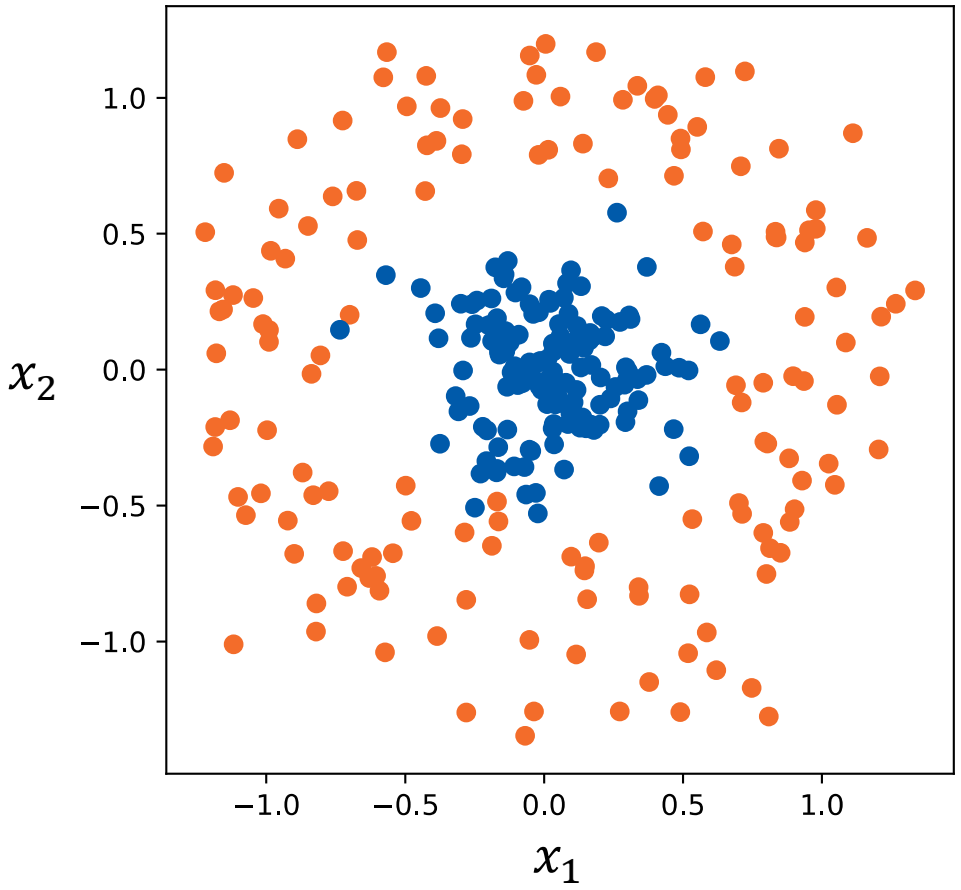
$$\hat{f}(x) = \mathbb{I}[\theta^{\mathrm{T}} x > 0]$$

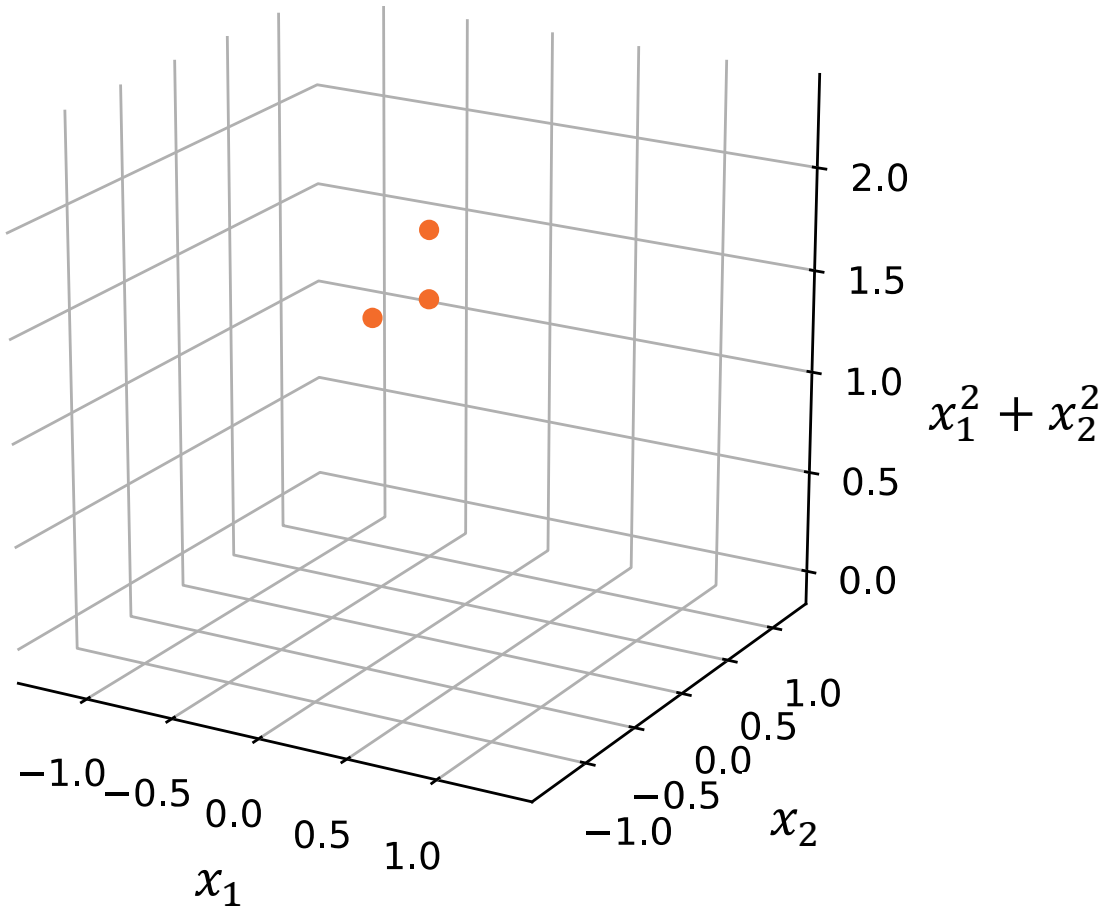


Targets $\in$ some finite set

Outputs linear in inputs

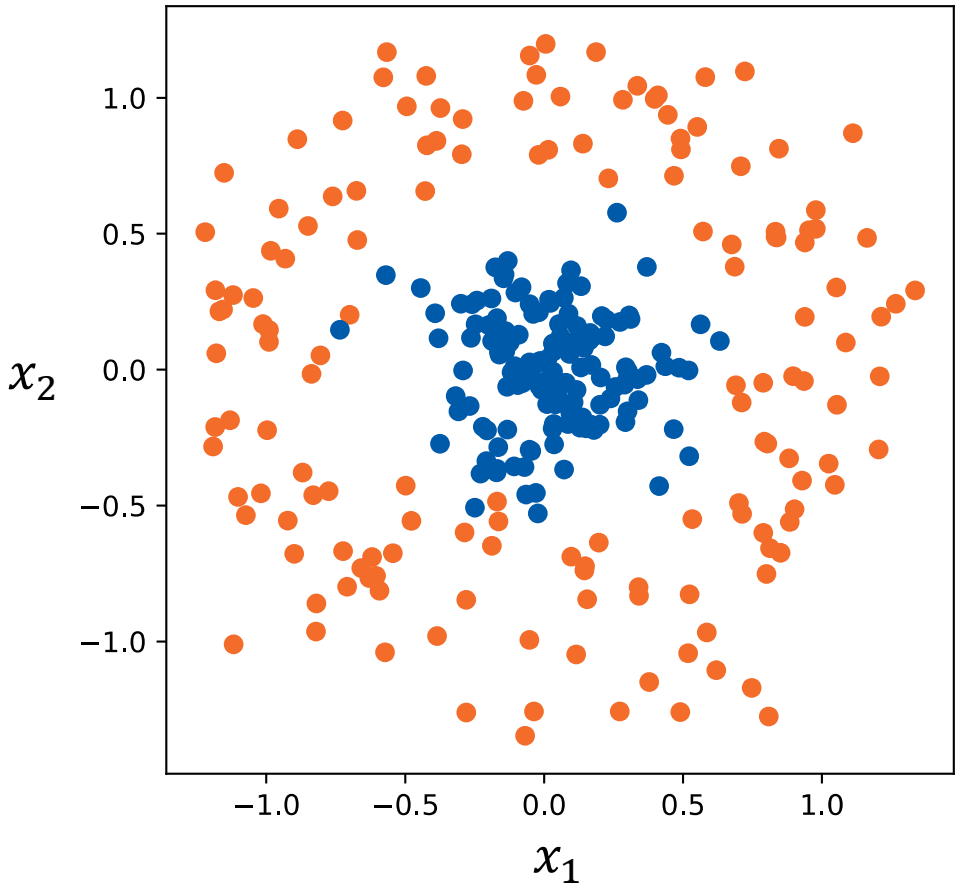
The hidden power

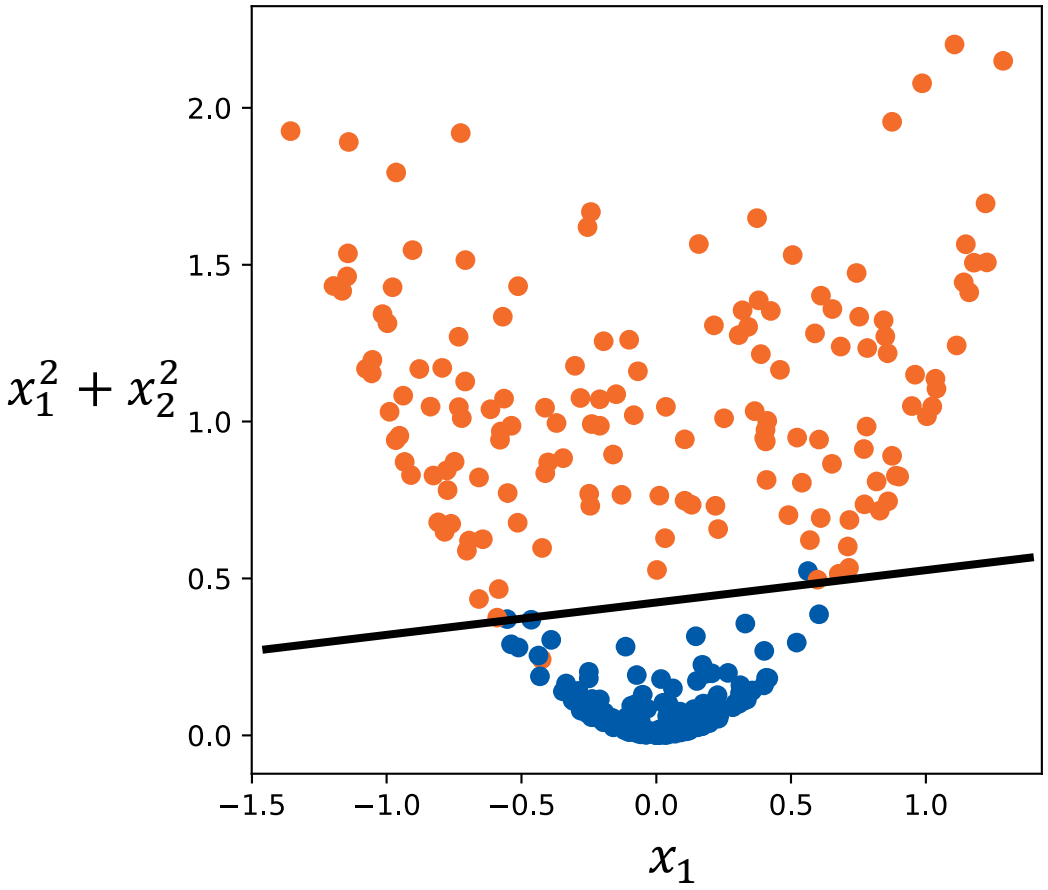




- Linearly **inseparable** ⟶ **separable** by **transforming the features**

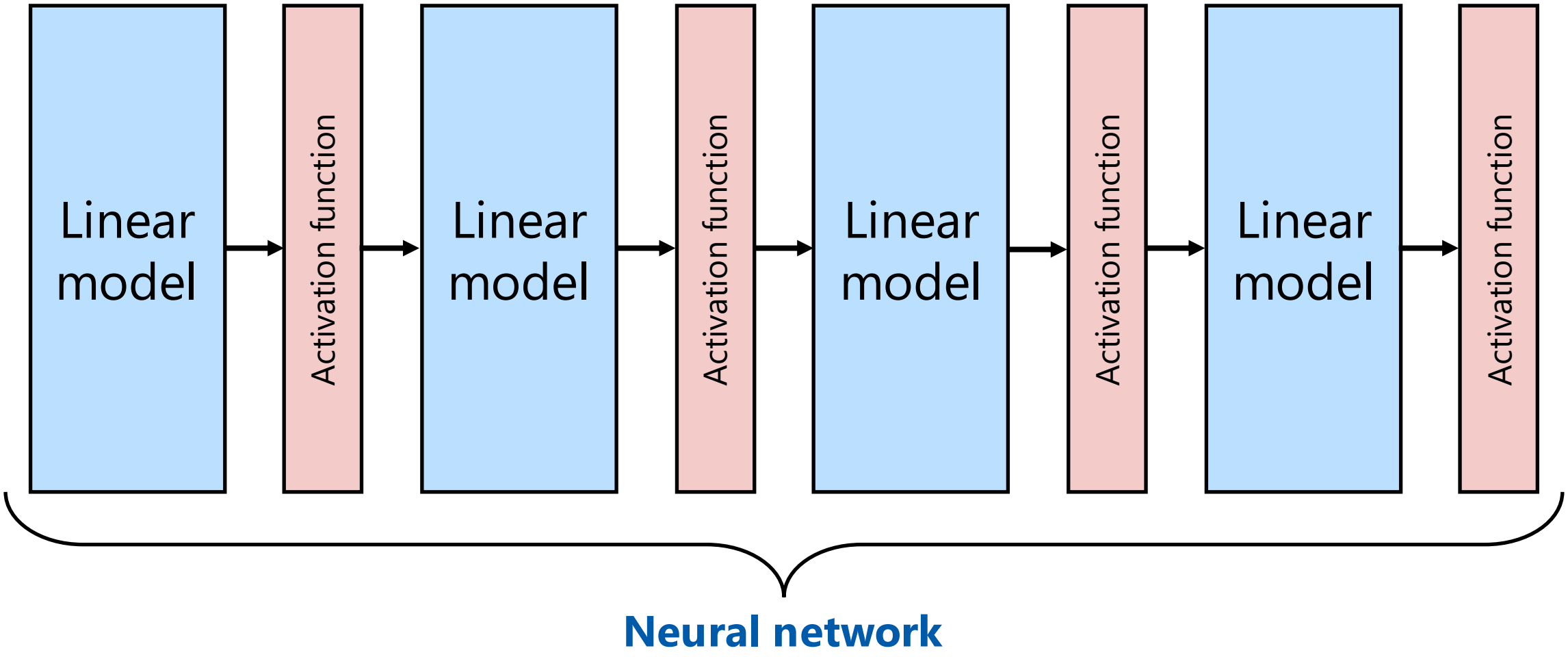
The hidden power





- Linearly **inseparable** ⟶ **separable** by **transforming the features**

Building block for deep models



- Better intuition for deep neural networks training

Interpretability

- One can take a look at the weights corresponding to each of the features separately, e.g.:

Note: this example is not based on real data, but rather taken from the lecturer's imagination

$$\textbf{PredictedIncome} = 1000 + 10 \times \textbf{Age} + 1000 \times \textbf{HasHigherEducation} - 0.1 \times \textbf{DistanceFromCapital}$$

- Higher weight means more impact on the prediction
  - Note: features need to be of same scale if you want to compare their weights

Learning check

1. A linear model has $\theta = (2, 3)$. For the object $x = (1, 4)$ it predicts $\theta^T x = \ldots$

a. 5

b. 9

c. 14

d. 24

2. PredictedIncome = 1000 + 10 × Age + 1000 × HasHigherEducation − 0.1 × DistanceFromCapital. A person gets higher education, everything else stays the same. The prediction changes by …

a. +10

b. +1000

c. −0.1

d. 0

3. Same model: Age = 30, HasHigherEducation = 1, DistanceFromCapital = 0. The predicted income is …

a. 2 300

b. 1 310

c. 2 030

d. 1 000

4. Why can we not compare the weights of Age and DistanceFromCapital directly?

a. the model is not linear

b. weights are random

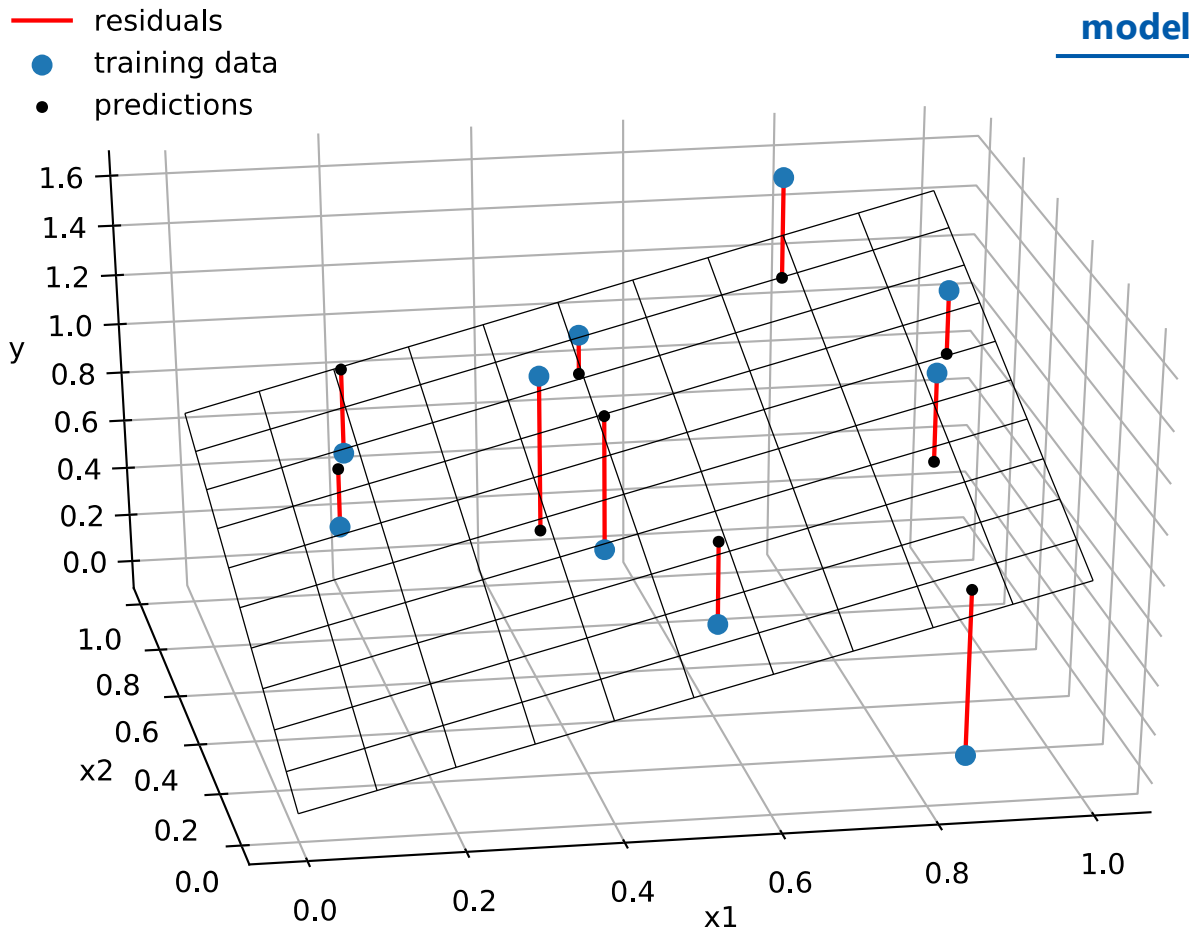
c. we always can

d. the features have different scales (years vs km)

Linear Regression



Linear Regression model



model prediction

$$\hat{f}_\theta(x) = \theta^{\mathrm{T}} x$$

parameters vector

features vector

$$\theta \in \mathbb{R}^d$$

$$x \in \mathcal{X} \subset \mathbb{R}^d$$

$$\frac{1}{N} \sum_{i=1 \ldots N} \left(y_i - \hat{f}_\theta(x_i) \right)^2 \longrightarrow \min_\theta$$

Mean Squared Error (MSE loss)

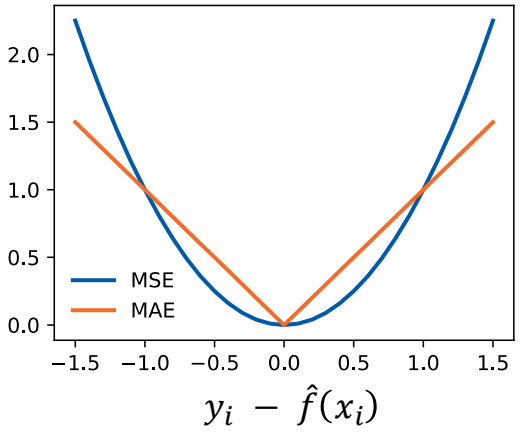
Common loss functions

Mean squared error (MSE): $$\frac{1}{N}\sum_{i=1\dots N}\left(y_i - \hat{f}_\theta(x_i)\right)^2$$

Common loss functions

Mean squared error (MSE): $$\frac{1}{N}\sum_{i=1\dots N}\left(y_i - \hat{f}_\theta(x_i)\right)^2$$

Mean absolute error (MAE): $$\frac{1}{N}\sum_{i=1\dots N}\left|y_i - \hat{f}_\theta(x_i)\right|$$



Common loss functions

Mean squared error (MSE):

$$\frac{1}{N}\sum_{i=1\dots N}\left(y_i - \hat{f}_\theta(x_i)\right)^2$$

Mean absolute error (MAE):

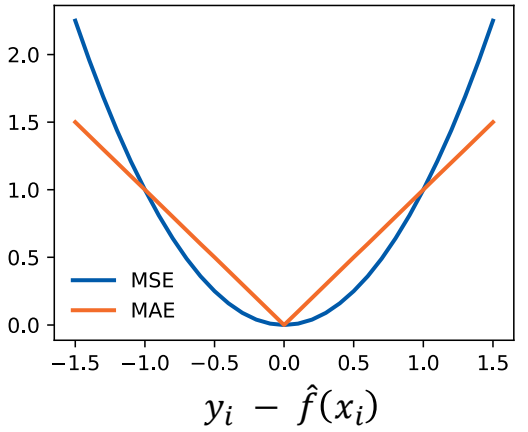
$$\frac{1}{N}\sum_{i=1\dots N}\left|y_i - \hat{f}_\theta(x_i)\right|$$

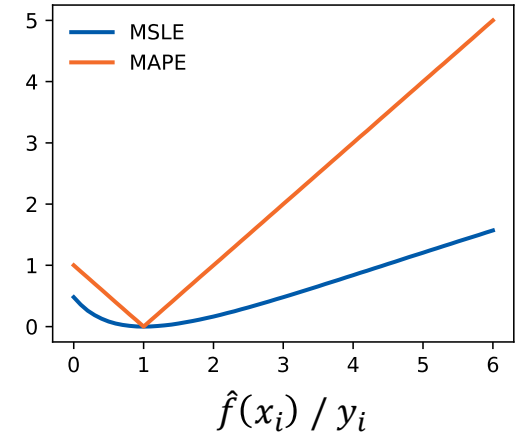
Mean absolute percentage error (MAPE):

$$\frac{1}{N}\sum_{i=1\dots N}\left|\frac{y_i - \hat{f}_\theta(x_i)}{y_i}\right|$$

Mean squared logarithmic error (MSLE):

$$\frac{1}{N}\sum_{i=1\dots N}\left(\log(y_i+1) - \log\left(\hat{f}_\theta(x_i)+1\right)\right)^2$$





Common loss functions

Mean squared error (MSE):

$$\frac{1}{N}\sum_{i=1\dots N}\left(y_i - \hat{f}_\theta(x_i)\right)^2$$

Mean absolute error (MAE):

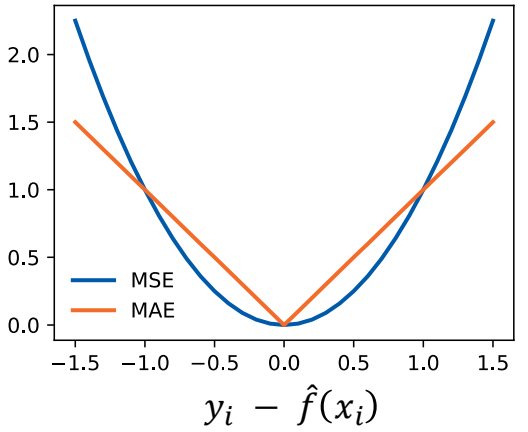
$$\frac{1}{N}\sum_{i=1\dots N}\left|y_i - \hat{f}_\theta(x_i)\right|$$

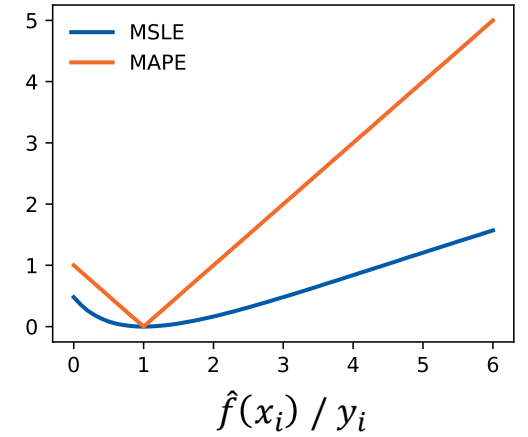
Mean absolute percentage error (MAPE):

$$\frac{1}{N}\sum_{i=1\dots N}\left|\frac{y_i - \hat{f}_\theta(x_i)}{y_i}\right|$$

Mean squared logarithmic error (MSLE):

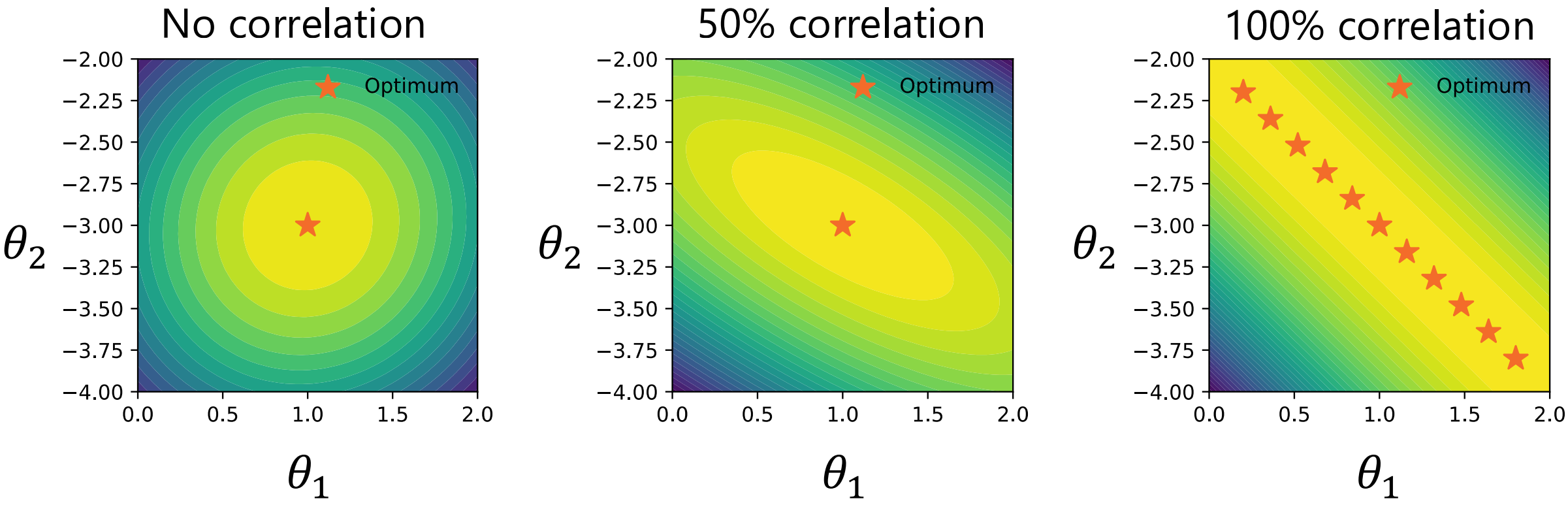
$$\frac{1}{N}\sum_{i=1\dots N}\left(\log(y_i+1) - \log\left(\hat{f}_\theta(x_i)+1\right)\right)^2$$





- Different **loss functions** also are related to different **assumptions about the data**

Feature correlations matter!



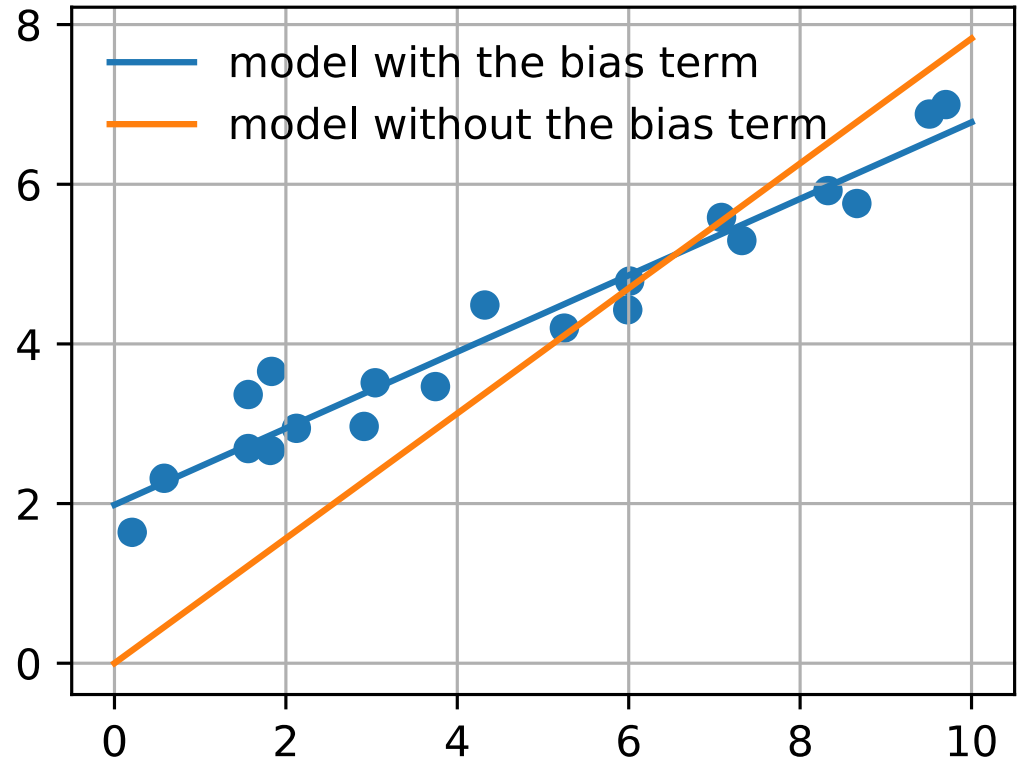
MSE level maps

Bias term

a.k.a. intercept term

$$\hat{f}_\theta(x) = \theta^{\mathrm{T}} x + \theta_0$$

$$\theta \in \mathbb{R}^d$$
$$\theta_0 \in \mathbb{R}$$
$$x \in \mathcal{X} \subset \mathbb{R}^d$$



- No need to redo the math – just add a **constant feature** to the design matrix:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \ddots & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^d \end{bmatrix} \longrightarrow X = \begin{bmatrix} 1 & x_1^1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N^1 & x_N^2 & \cdots & x_N^d \end{bmatrix}$$

Learning check

1. True targets $y = (2, 4)$, predictions $\hat{y} = (3, 2)$. The MSE is …

a. 1.5

b. 2.5

c. 5

d. 3

2. Same data. The MAE is …

a. 1.5

b. 2.5

c. 3

d. 0.5

3. Two features are 100% correlated. The minimum of the MSE is then …

a. one single point

b. always at $\theta = 0$

c. does not exist at all

d. not unique: a whole line of equally good θ

4. Why do we add a column of ones to the design matrix?

a. to get the bias (intercept) term θ_0

b. to make the matrix square

c. to remove correlations

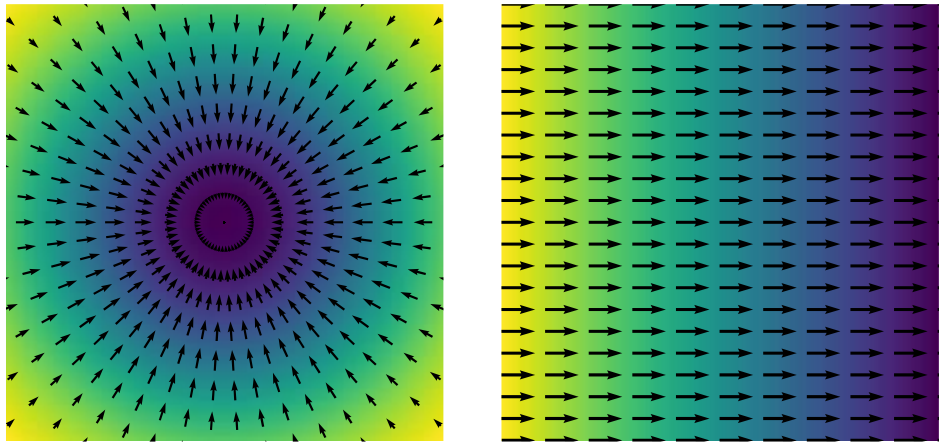
d. to speed up training

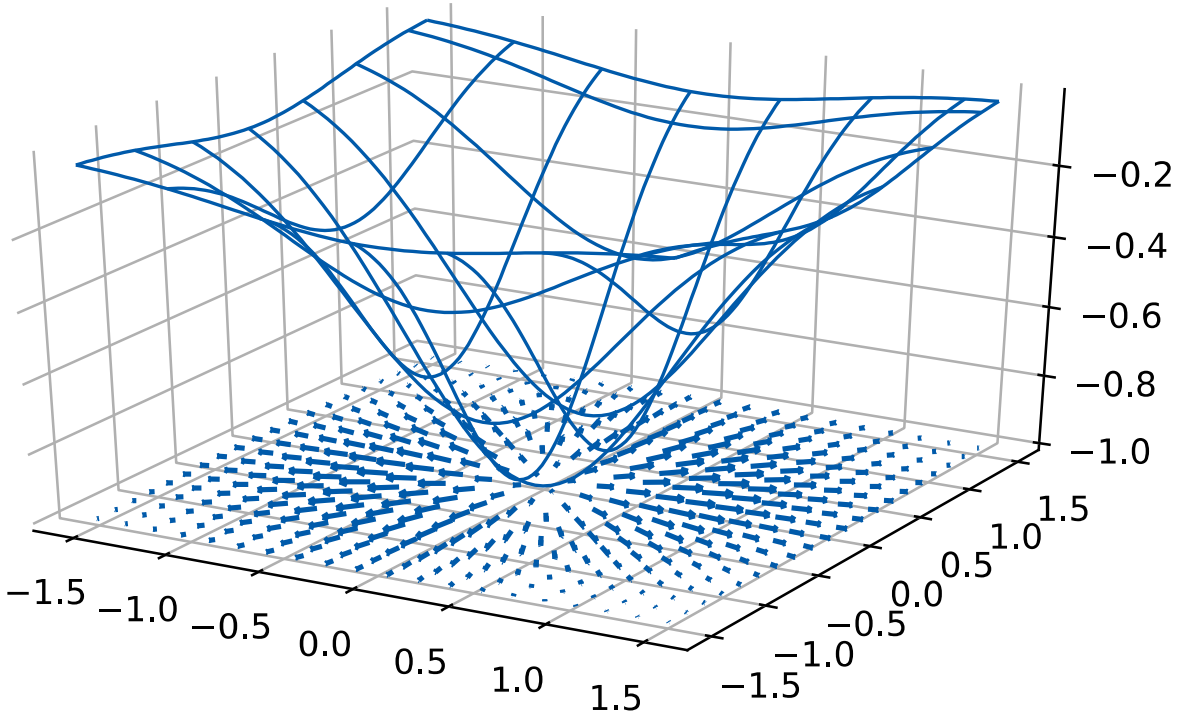
Numerical & Stochastic Optimization



Gradient

- Gradient: $\nabla_x f(x) \equiv \left(\frac{\partial f(x)}{\partial x_1}, \ldots, \frac{\partial f(x)}{\partial x_d}\right)$
- Points towards **steepest function increase**





Gradient Descent Optimization

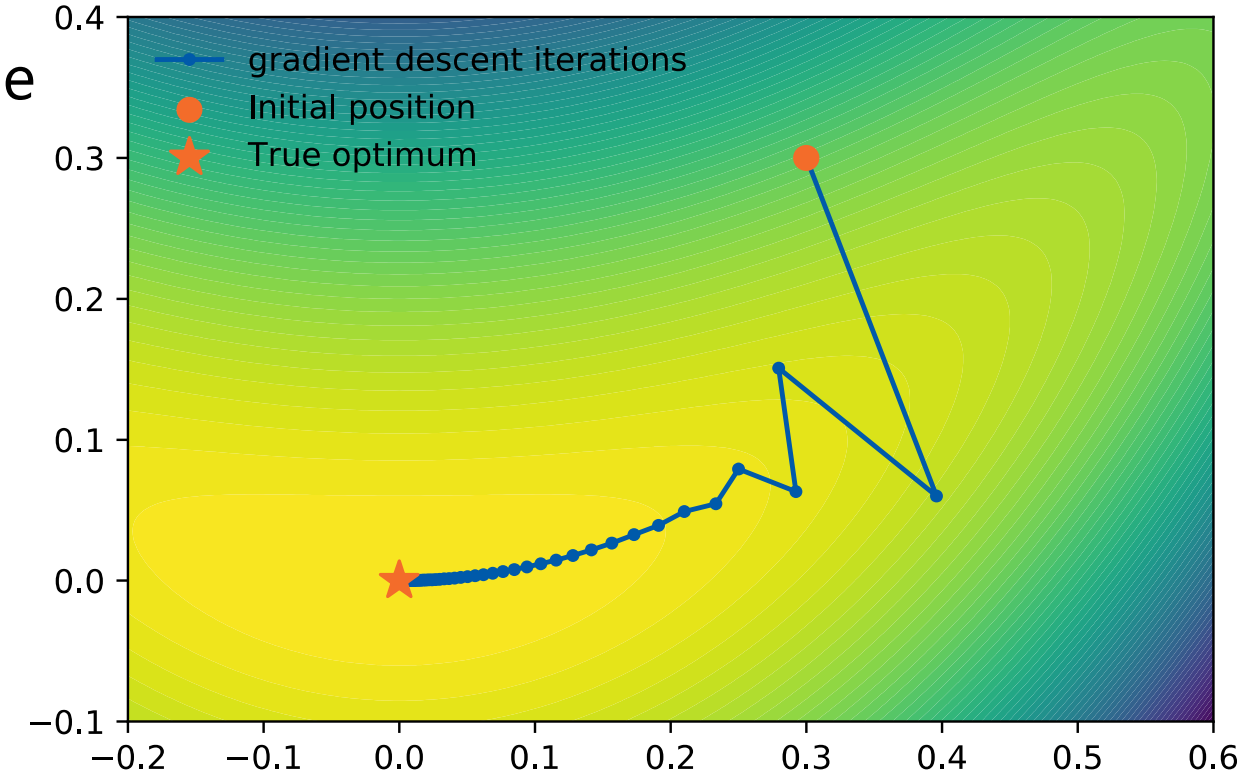
- Can optimize functions starting at some initial point $x^{(0)}$ and moving opposite to the gradient:

$$x^{(k)} \leftarrow x^{(k-1)} - \alpha \nabla_x f\left(x^{(k-1)}\right)$$

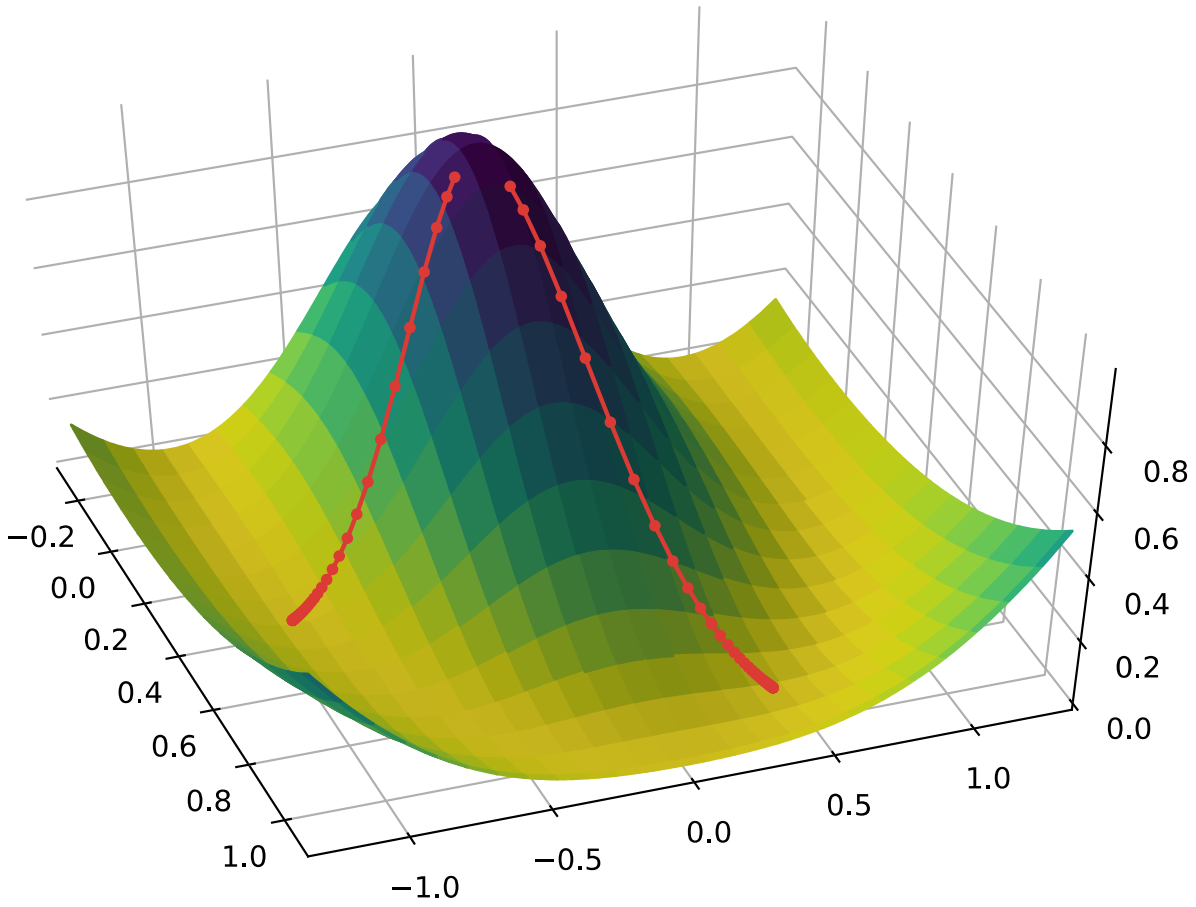
with $\alpha \in \mathbb{R},\ \alpha > 0$ – learning rate.

- For smooth **convex** functions with a single minimum x^*:

$$f\left(x^{(k)}\right) - f(x^*) = \mathcal{O}\left(\frac{1}{k}\right)$$

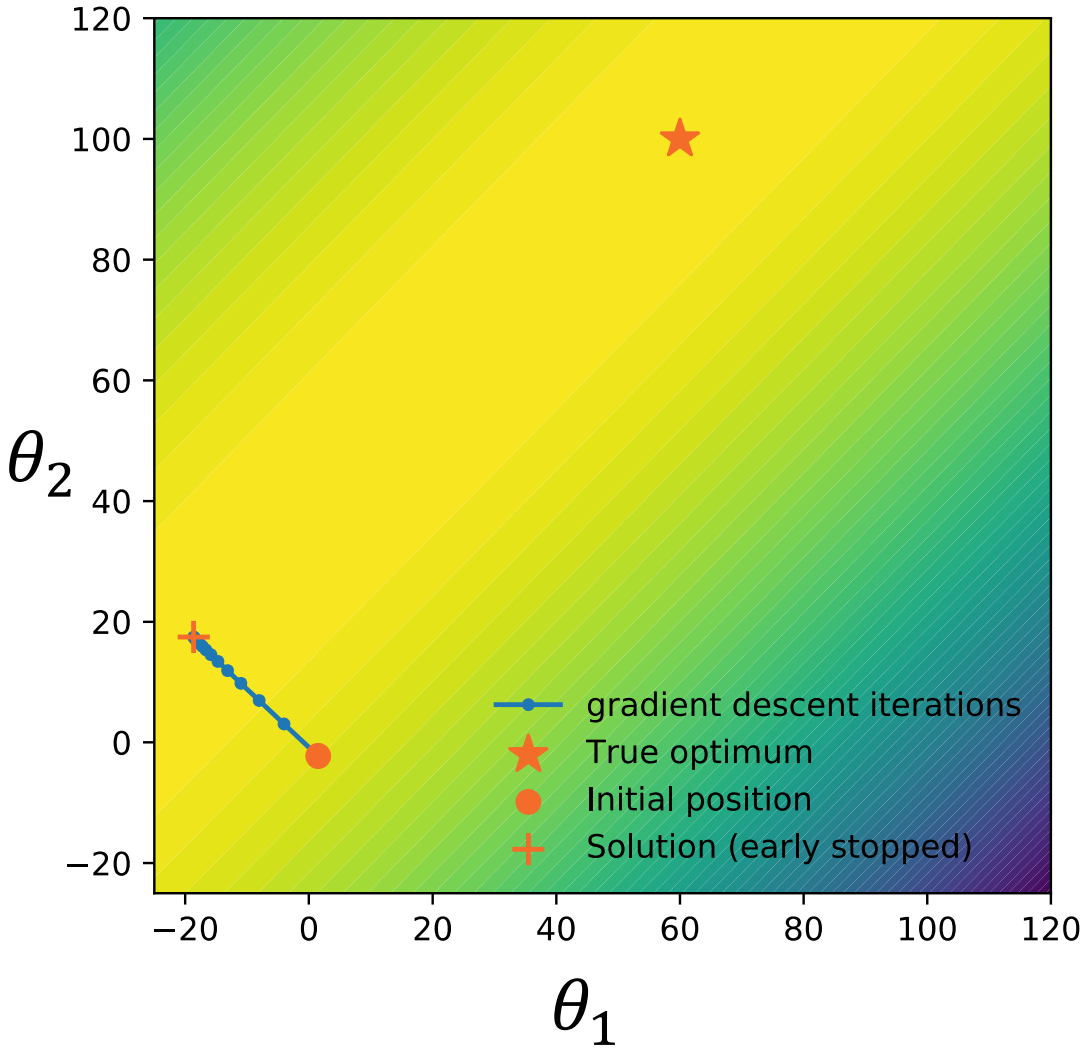


Gradient descent for non-convex functions



- May get to a minimum which is not global
- Result depends on the starting point

Gradient descent as means for regularisation



- Large parameter values typically mean overfitting
- You may avoid this problem by initializing parameters with **small values** and **early stopping** the gradient descent

Stochastic Gradient Descent (SGD)

- In machine learning we optimize loss functions which are typically averages over objects:

$$L = \frac{1}{N} \sum_{i = 1 \ldots N} \mathcal{L}\left(y_i, \widehat{f_\theta}(x_i)\right)$$

Stochastic Gradient Descent (SGD)

- In machine learning we optimize loss functions which are typically averages over objects:

$$L = \frac{1}{N} \sum_{i = 1 \ldots N} \mathcal{L}\left(y_i, \widehat{f_\theta}(x_i)\right)$$

- For large N, gradient descent is computationally inefficient and may be unfeasible in terms of memory consumption

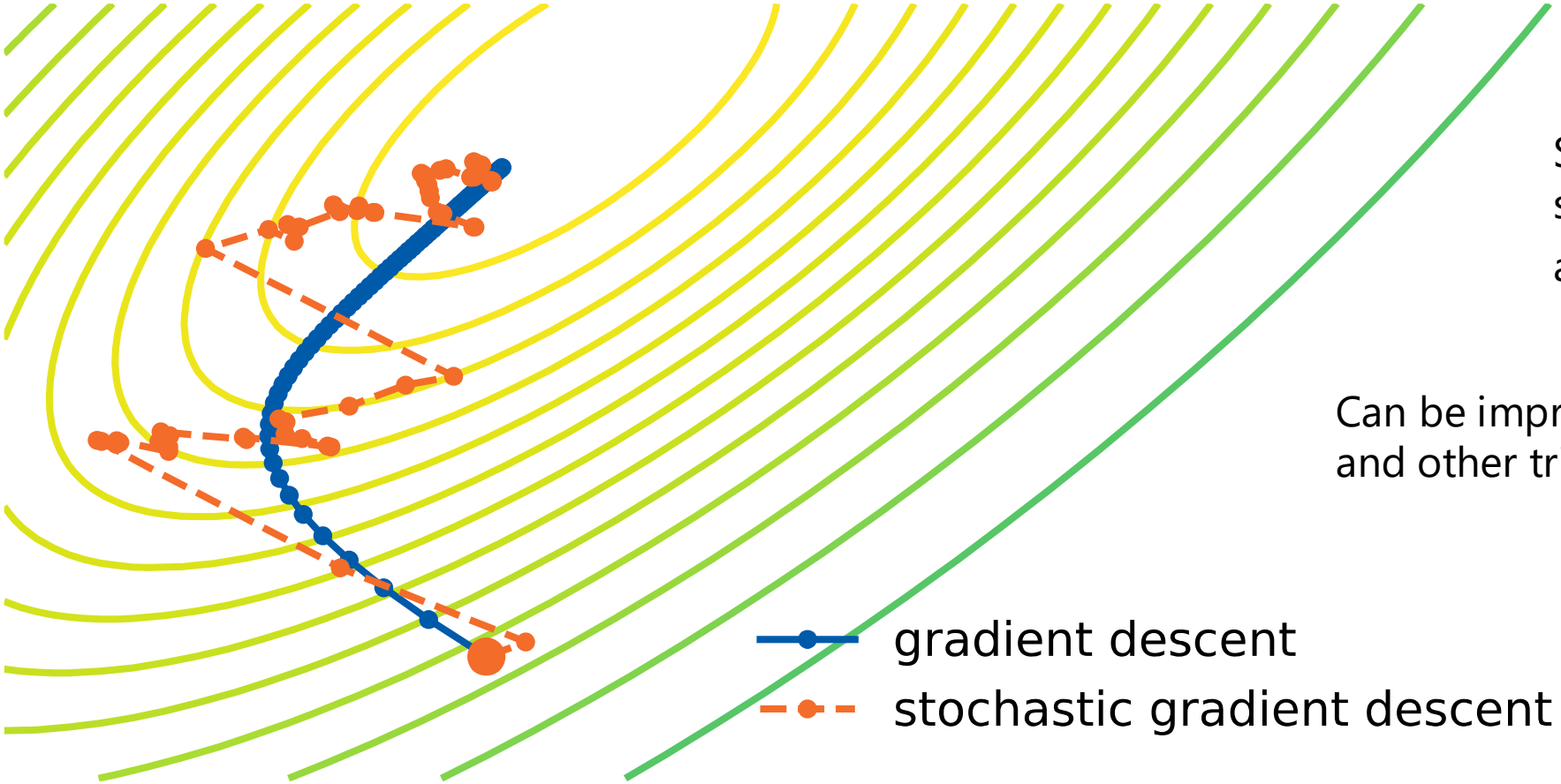
Stochastic Gradient Descent (SGD)

- In machine learning we optimize loss functions which are typically averages over objects:

$$L = \frac{1}{N} \sum_{i = 1 \ldots N} \mathcal{L}\left(y_i, \widehat{f_\theta}(x_i)\right)$$

- For large N, gradient descent is computationally inefficient and may be unfeasible in terms of memory consumption
- Alternative:
  - At each step k pick $l_k \in \{1, \ldots, N\}$ at random
  - Optimize: $\theta^{(k)} \leftarrow \theta^{(k-1)} - \alpha \nabla_\theta \mathcal{L}\left(y_{l_k}, \widehat{f_\theta}(x_{l_k})\right)\Big|_{\theta = \theta^{(k-1)}}$

Stochastic Gradient Descent (SGD)



SGD convergence rate for smooth **convex** functions with a single minimum: $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$

Can be improved by batching and other tricks

Learning check

1. The gradient of a function points in the direction of …

a. the steepest decrease

b. the minimum

c. the steepest increase

d. zero

2. Gradient descent step: $x = 3$, gradient $= 4$, learning rate $\alpha = 0.5$. The new x is …

a. 5

b. 2

c. -1

d. 1

3. For a non-convex function, gradient descent …

a. may stop in a local minimum, depending on the starting point

b. always finds the global minimum

c. never moves

d. does not need a learning rate

4. At each step, stochastic gradient descent (SGD) uses …

a. all N objects

b. one randomly chosen object (or a small batch)

c. no data

d. only the first object

Feature Expansion



Feature expansion

- One can perform **feature transformations** with any function $\Phi\colon \mathbb{R}^d \to \mathbb{R}^{d'}$

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \ddots & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^d \end{bmatrix} \longrightarrow \Phi(X) = \begin{bmatrix} \Phi^1(x_1^1, \dots, x_1^d) & \cdots & \Phi^{d'}(x_1^1, \dots, x_1^d) \\ \Phi^1(x_2^1, \dots, x_2^d) & \cdots & \Phi^{d'}(x_2^1, \dots, x_2^d) \\ \vdots & \ddots & \vdots \\ \Phi^1(x_N^1, \dots, x_N^d) & \cdots & \Phi^{d'}(x_N^1, \dots, x_N^d) \end{bmatrix}$$

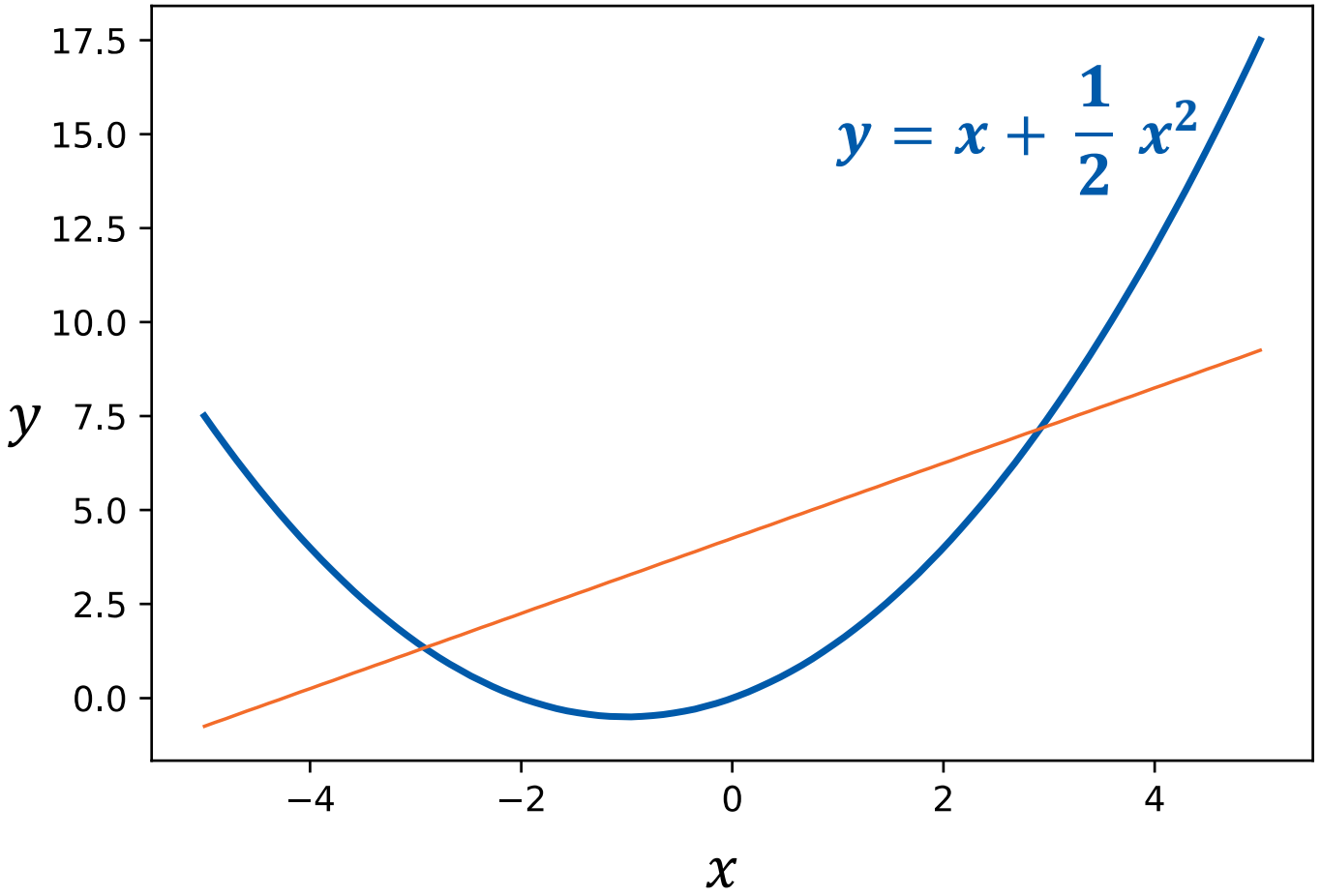
Feature expansion

- One can perform **feature transformations** with any function Φ: $\mathbb{R}^d \to \mathbb{R}^{d'}$

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \ddots & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^d \end{bmatrix} \longrightarrow \Phi(X) = \begin{bmatrix} \Phi^1(x_1^1, \dots, x_1^d) & \cdots & \Phi^{d'}(x_1^1, \dots, x_1^d) \\ \Phi^1(x_2^1, \dots, x_2^d) & \cdots & \Phi^{d'}(x_2^1, \dots, x_2^d) \\ \vdots & \ddots & \vdots \\ \Phi^1(x_N^1, \dots, x_N^d) & \cdots & \Phi^{d'}(x_N^1, \dots, x_N^d) \end{bmatrix}$$

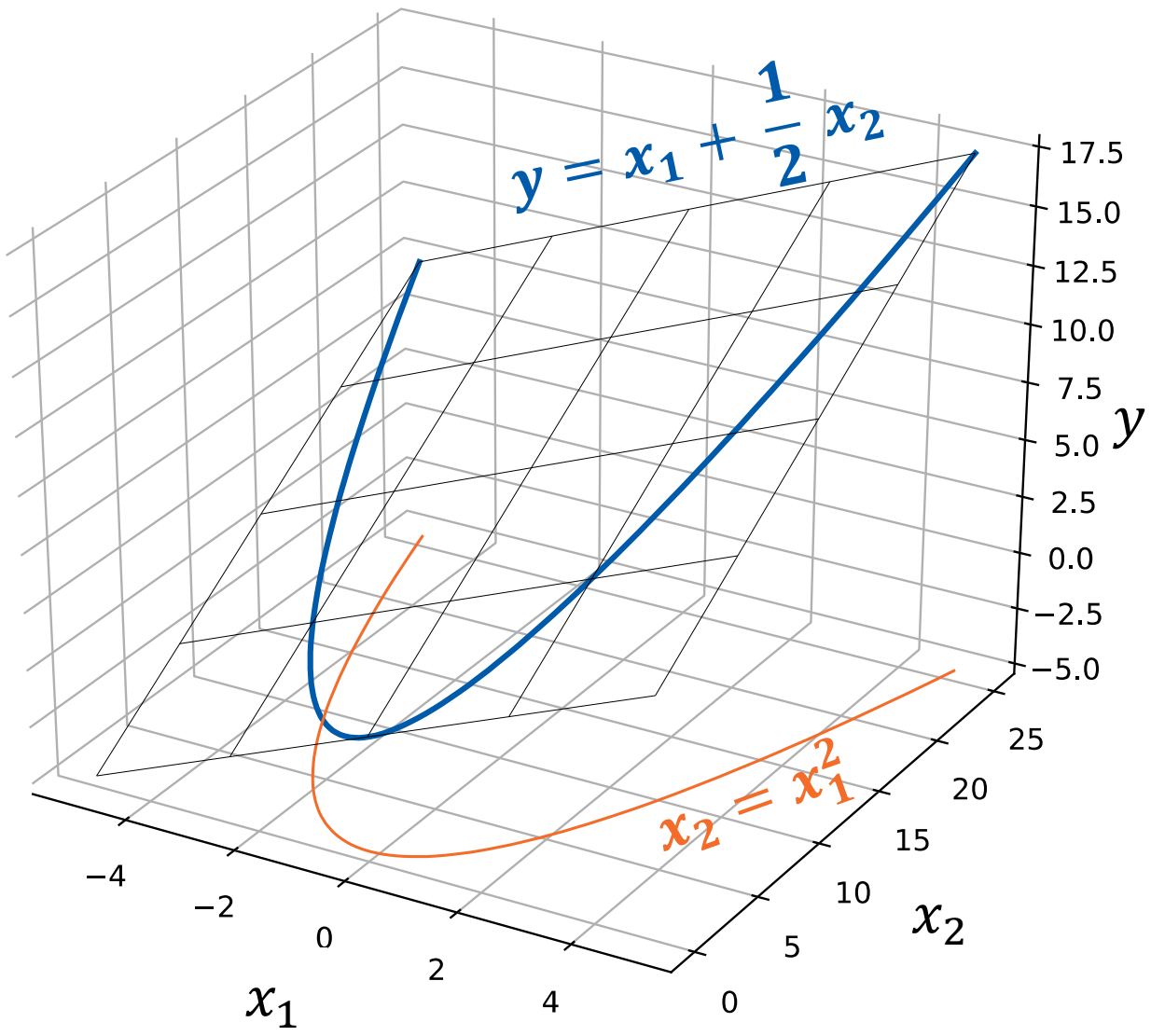
- Finding the best function Φ is called **feature engineering**
  - It is an important part of machine learning and requires deep understanding of the underlying problem and the data

Example: polynomial features



- Can't be solved with the only linear feature (x)

Example: polynomial features



- Introducing another feature does the job:

$$(x_1, x_2) \equiv (x, x^2)$$

- Now our estimate is:

$$\hat{f}(x) = \theta_1 x + \theta_2 x^2$$

Polynomial features of degree p (general case)

For the original features:

$$\left(x_i^1, x_i^2, \dots, x_i^d \right)$$

introduce all unique multiplicative combinations of the form:

$$\left(x_i^{k_1}\right)^{p_1} \cdot \left(x_i^{k_2}\right)^{p_2} \cdot \dots \cdot \left(x_i^{k_m}\right)^{p_m}$$

with $p_1 + p_2 + \dots + p_m \leq p$

Example: degree 3 polynomial features

For the original features (a, b, c):

$$(1, a, b, c, a^2, ab, ac, b^2, bc, c^2, a^3, a^2b, a^2c, ab^2, abc, ac^2, b^3, b^2c, bc^2, c^3)$$

Learning check

1. The data follow $y = x + \frac{1}{2}x^2$. A linear regression with the single feature x …

a. fits the data perfectly

b. has MSE = 0

c. cannot be trained

d. cannot fit the curve well

2. Which extra feature lets a linear model fit $y = x + \frac{1}{2}x^2$ exactly?

a. x^3

b. log x

c. x^2

d. 1 / x

3. The model $\theta_1 x + \theta_2 x^2$ is a curve in x. It is still called linear because it is linear in …

a. the parameters θ

b. x

c. y

d. nothing

4. Degree-2 polynomial features of one feature x (with the bias) are …

a. (1, x)

b. $(1, x, x^2)$

c. (x^2)

d. $(1, x, x^2, x^3)$

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization
- Linear Regression with MSE loss allows for **analytical solution**

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization
- Linear Regression with MSE loss allows for **analytical solution**
- The stability of the solution depends on the **feature correlations**

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization
- Linear Regression with MSE loss allows for **analytical solution**
- The stability of the solution depends on the **feature correlations**
- Linear models can be optimized with gradient descent and stochastic gradient descent

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization
- Linear Regression with MSE loss allows for **analytical solution**
- The stability of the solution depends on the **feature correlations**
- Linear models can be optimized with gradient descent and stochastic gradient descent
  - In some cases this can **regularize** the solution
- **Feature transformations** allow for very powerful use of the linear models

Summary

- Understanding linear models gives **useful insights** into more complicated machine learning algorithms and optimization
- Linear Regression with MSE loss allows for **analytical solution**
- The stability of the solution depends on the **feature correlations**
- Linear models can be optimized with gradient descent and stochastic gradient descent
  - In some cases this can **regularize** the solution
- **Feature transformations** allow for very powerful use of the linear models
- Food for thought: how does polynomial feature expansion affect the complexity of the model?